Category C Assignment – Solutions

Arrays, loops and conditionals · Java · all code compiled and run against the sample inputs

#	Problem	Technique	Time	Extra space
A1	Product of Array Except Self	Prefix pass + suffix pass	O(n)	O(1) beyond output
A2	Maximum Subarray	Kadane's algorithm	O(n)	O(1)
A3	3Sum	Sort + two pointers	$O(n^2)$	O(1) beyond output
A4	Subarray Sum Equals K	Prefix sums + hash map	O(n)	O(n)
A5	Find Minimum in Rotated Sorted Array	Modified binary search	O(log n)	O(1)

How to run

All five methods plus a `main` that prints the sample outputs are in `Solutions.java`. Compile with `javac Solutions.java` and run with `java Solutions`.

Each following page covers one problem: approach, Java code, a dry run, test results against the sample inputs, pitfalls and complexity.

A1. Product of Array Except Self

Approach

answer[i] equals (product of everything left of i) × (product of everything right of i). Build the left products directly in the output array, then sweep from the right with one running variable. Division is never used, so zeros are handled automatically.

- **Forward pass:** answer[0] = 1; answer[i] = answer[i-1] × nums[i-1].
- **Backward pass:** keep `right` = 1; for i from n-1 down to 0, multiply answer[i] by right, then right ×= nums[i].

Java solution

```
static int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] answer = new int[n];
    answer[0] = 1;
    for (int i = 1; i < n; i++) {
        answer[i] = answer[i - 1] * nums[i - 1];   // product of everything left of i
    }
    int right = 1;
    for (int i = n - 1; i >= 0; i--) {
        answer[i] *= right;                        // multiply in product right of i
        right *= nums[i];
    }
    return answer;
}
```

Dry run

i	left product (after pass 1)	right product	answer[i]
0	1	2×3×4 = 24	24
1	1	3×4 = 12	12
2	2	4	8
3	6	1	6

Test results

Input	Output
[1, 2, 3, 4]	[24, 12, 8, 6] (matches)
[-1, 1, 0, -3, 3]	[0, 0, 9, 0, 0] (matches)

Edge cases and pitfalls

- A single zero makes every other position 0; the zero's own slot gets the product of the rest (9 in the sample).
- Two zeros make the whole output 0, which this method produces naturally.
- Do not allocate separate left[] and right[] arrays if you want O(1) extra space.

Complexity

Time O(n) with two passes; extra space O(1) apart from the output array.

A2. Maximum Subarray

Approach

At each element decide: extend the running subarray (current + x) or restart at x. Keep whichever is larger, and track the best value seen so far.

- Initialise current = best = nums[0] (not 0, so all-negative arrays work).
- For each next x: current = max(x, current + x); best = max(best, current).

Java solution

```
static int maxSubArray(int[] nums) {
    int current = nums[0];
    int best = nums[0];
    for (int i = 1; i < nums.length; i++) {
        current = Math.max(nums[i], current + nums[i]);  // restart vs extend
        best = Math.max(best, current);
    }
    return best;
}
```

Dry run

x	current	best
-2	-2	-2
1	1	1
-3	-2	1
4	4	4
-1	3	4
2	5	5
1	6	6
-5	1	6
4	5	6

Test results

Input	Output
[-2, 1, -3, 4, -1, 2, 1, -5, 4]	6 (matches; subarray [4, -1, 2, 1])
[-3, -1, -2]	-1 (matches)

Edge cases and pitfalls

- Starting best at 0 is wrong for all-negative arrays; the answer must contain at least one element.
- Divide-and-conquer follow-up: split at mid, answer = max(left best, right best, best crossing the middle). That is O(n log n).

Complexity

Time O(n), extra space O(1).

A3. 3Sum

Approach

Sort the array. Fix the first element a[i], then search the remaining part with two pointers (lo, hi) for pairs that sum to -a[i]. Skip duplicate values at all three levels so no triplet repeats.

- Sort a copy of the array. Stop when a[i] > 0 (three positives cannot sum to 0).
- Skip i if a[i] equals a[i-1].
- Move lo up if the sum is negative, hi down if positive. On a match, record the triplet, then skip equal values on both sides before moving both pointers.

Java solution

```
static int[][] threeSum(int[] nums) {
    int[] a = nums.clone();
    Arrays.sort(a);
    List<int[]> result = new ArrayList<>();
    int n = a.length;
    for (int i = 0; i < n - 2; i++) {
        if (a[i] > 0) break;                        // no way to reach zero
        if (i > 0 && a[i] == a[i - 1]) continue;    // skip duplicate anchor
        int lo = i + 1, hi = n - 1;
        while (lo < hi) {
            int sum = a[i] + a[lo] + a[hi];
            if (sum == 0) {
                result.add(new int[]{a[i], a[lo], a[hi]});
                while (lo < hi && a[lo] == a[lo + 1]) lo++;   // skip duplicate lo
                while (lo < hi && a[hi] == a[hi - 1]) hi--;   // skip duplicate hi
                lo++;
                hi--;
            } else if (sum < 0) {
                lo++;
            } else {
                hi--;
            }
        }
    }
    return result.toArray(new int[0][]);
}
```

Dry run

Sorted: [-4, -1, -1, 0, 1, 2]	Action
i=0 (-4)	No pair reaches 4; nothing found
i=1 (-1)	lo=-1, hi=2 → sum 0 → record [-1, -1, 2]; lo=0, hi=1 → sum 0 → record [-1, 0, 1]
i=2 (-1)	Duplicate of previous anchor → skipped
i=3 (0)	Positive-side values only; nothing found

Test results

Input	Output
[-1, 0, 1, 2, -1, -4]	[[-1, -1, 2], [-1, 0, 1]] (matches)
[0, 0, 0]	[[0, 0, 0]] (matches, reported once)

Edge cases and pitfalls

- Forgetting to skip after a match gives repeated triplets.
- Use `lo < hi` inside the skip loops so the pointers never cross.
- Sorting a clone keeps the caller's array unchanged.

Complexity

Sort O(n log n) + n two-pointer scans of O(n) = O(n^2) time; O(1) extra space besides the result list.

A4. Subarray Sum Equals K

Approach

A subarray (i, j] has sum prefix[j] - prefix[i]. So at each position, the number of subarrays ending here with sum k equals the number of earlier prefixes equal to (currentSum - k). A hash map stores how often each prefix sum has appeared.

- Put (0 → 1) in the map first. This represents the empty prefix so subarrays starting at index 0 are counted.
- For each x: sum += x; count += map[sum - k]; then map[sum]++ (update after counting).
- A sliding window fails here because negative numbers mean extending the window does not always increase the sum.

Java solution

```
static int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    freq.put(0, 1);                                    // empty prefix
    int sum = 0, count = 0;
    for (int x : nums) {
        sum += x;
        count += freq.getOrDefault(sum - k, 0);
        freq.put(sum, freq.getOrDefault(sum, 0) + 1);
    }
    return count;
}
```

Dry run

x	sum	sum - k (k=0)	map hits	count
1	1	1	0	0
-1	0	0	1 (empty prefix)	1
0	0	0	2 (two earlier zeros)	3

Test results

Input	Output
[1, 1, 1], k = 2	2 (matches)
[1, -1, 0], k = 0	3 (matches)

Edge cases and pitfalls

- Missing the (0 → 1) base case undercounts subarrays that begin at index 0.
- Update the map after adding to count, otherwise k = 0 counts empty subarrays.

Complexity

Time O(n), space O(n) for the prefix-sum frequency map.

A5. Find Minimum in Rotated Sorted Array

Approach

Compare nums[mid] with the rightmost element nums[hi]. If nums[mid] > nums[hi], the drop (the minimum) lies strictly to the right of mid. Otherwise the right half is sorted and the minimum is at mid or to its left.

- lo = 0, hi = n-1. While lo < hi: mid = lo + (hi-lo)/2.
- If nums[mid] > nums[hi] then lo = mid + 1, else hi = mid.
- When lo meets hi, nums[lo] is the minimum. An unrotated array shrinks to index 0 naturally.

Java solution

```
static int findMin(int[] nums) {
    int lo = 0, hi = nums.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] > nums[hi]) {
            lo = mid + 1;                          // min is in right half
        } else {
            hi = mid;                              // min is mid or in left half
        }
    }
    return nums[lo];
}
```

Dry run

lo	hi	mid	nums[mid] vs nums[hi]	move
0	6	3	7 > 2	lo = 4
4	6	5	1 < 2	hi = 5
4	5	4	0 < 1	hi = 4
4	4	-	stop	answer nums[4] = 0

Test results

Input	Output
[3, 4, 5, 1, 2]	1 (matches)
[4, 5, 6, 7, 0, 1, 2]	0 (matches)
[11, 13, 15, 17]	11 (matches, no rotation)

Edge cases and pitfalls

- Compare against nums[hi], not nums[lo]; comparing with lo breaks the no-rotation case.
- Use hi = mid (not mid - 1) so the minimum is never discarded.
- This version assumes distinct elements, as the problem guarantees.

Complexity

Time O(log n), space O(1).